NAME: janani.k

REG.NO:192511243

COURSE CODE: CSA0503

COURSE NAME: DATABASE MANAGEMENT SYSTEMS

1. Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.

Sol:

Employee Table

EmpID	EmpName	Salary	DeptID
101	John	50000	D1
102	Alice	65000	D2
103	David	45000	D1
104	Emma	70000	D3

Department Table

DeptID	DeptName
D1	HR
D2	IT
D3	Finance

1. Selection (σ)

Retrieves rows satisfying a condition.

Query

Employees having salary greater than 50000.

Relational Algebra

σ Salary > 50000 (Employee)

EmpID	EmpName	Salary	DeptID
102	Alice	65000	D2
104	Emma	70000	D3

2. Projection (π) : Retrieves selected columns only.

Query : Display employee names.

Relational Algebra

π EmpName(Employee)

EmpName
John
Alice
David
Emma

3. Union (∪)

Suppose another relation exists.

EmpID	EmpName
105	Chris
106	Sophia

4. Relational Algebra

π EmpName(Employee)
∪
π EmpName(Employee_New)

5. Cartesian Product (×)

Combine every employee with every department.

Relational Algebra

Employee × Department

If Employee has 4 records and Department has 3 records,

Total tuples = 4 × 3 = 12

6. Join (⋈)

Display employee names with department names.

Relational Algebra

Employee ⋈ Employee.DeptID = Department.DeptID Department

Employee Department
John HR
Alice IT
David HR
Emma Finance

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

Sol:

```
mysql> CREATE TABLE Department(
    -> DeptID INT PRIMARY KEY,
    -> DeptName VARCHAR(50) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.21 sec)

mysql>
mysql> CREATE TABLE Student(
    -> SID INT PRIMARY KEY,
    -> Name VARCHAR(50) NOT NULL,
    -> DOB DATE NOT NULL,
    -> DeptID INT,
    -> FOREIGN KEY (DeptID)
    -> REFERENCES Department(DeptID)
    -> ON DELETE CASCADE
    -> ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> INSERT INTO Student
    -> VALUES
    -> (101,'Rahul','2004-01-12',1),
    -> (102,'Sneha','2004-08-25',2),
    -> (103,'Karan','2003-11-10',3);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> desc Student;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| SID    | int         | NO   | PRI | NULL    |       |
| Name   | varchar(50) | NO   |     | NULL    |       |
| DOB    | date        | NO   |     | NULL    |       |
| DeptID | int         | YES  | MUL | NULL    |       |
+--------+-------------+------+-----+---------+-------+
```

```
mysql> desc department;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptID   | int         | NO   | PRI | NULL    |       |
| DeptName | varchar(50) | NO   | UNI | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

DDL (Data Definition Language) is used to create and modify database objects. The CREATE TABLE statement defines the table structure, while constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE ensure data integrity and maintain relationships between tables.

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a Sales dataset.

Sol:

SaleID	Product	Department	Amount
1	Laptop	Electronics	60000
2	Mouse	Electronics	800
3	Chair	Furniture	5000
4	Table	Furniture	8000

SaleID Product Department Amount

5 Monitor Electronics 12000

```
mysql> CREATE TABLE Sales (
    ->     SaleID INT PRIMARY KEY,
    ->     Product VARCHAR(50) NOT NULL,
    ->     Department VARCHAR(50) NOT NULL,
    ->     Amount DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO Sales (SaleID, Product, Department, Amount)
    -> VALUES
    -> (1, 'Laptop', 'Electronics', 60000),
    -> (2, 'Mouse', 'Electronics', 800),
    -> (3, 'Chair', 'Furniture', 5000),
    -> (4, 'Table', 'Furniture', 8000),
    -> (5, 'Monitor', 'Electronics', 12000),
    -> (6, 'Keyboard', 'Electronics', 1500),
    -> (7, 'Sofa', 'Furniture', 25000),
    -> (8, 'Printer', 'Electronics', 18000);
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT COUNT(*) AS TotalSales
    -> FROM Sales;
+------------+
| TotalSales |
+------------+
|          8 |
+------------+
1 row in set (0.03 sec)
```

```
mysql> SELECT SUM(Amount) AS TotalSalesAmount
    -> FROM Sales;
+------------------+
| TotalSalesAmount |
+------------------+
|        130300.00 |
+------------------+
1 row in set (0.01 sec)

mysql> SELECT AVG(Amount) AS AverageSales
    -> FROM Sales;
+---------------+
| AverageSales |
+---------------+
| 16287.500000 |
+---------------+
1 row in set (0.01 sec)
```

```
mysql> SELECT MAX(Amount) AS HighestSale
    -> FROM Sales;
+-------------+
| HighestSale |
+-------------+
|    60000.00 |
+-------------+
1 row in set (0.01 sec)

mysql> SELECT MIN(Amount) AS LowestSale
    -> FROM Sales;
+------------+
| LowestSale |
+------------+
|     800.00 |
+------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT Department,
    ->        COUNT(*) AS NumberOfSales,
    ->        SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Department;
+------------+---------------+-------------+
| Department | NumberOfSales | TotalAmount |
+------------+---------------+-------------+
| Electronics |             5 |    92300.00 |
| Furniture  |             3 |    38000.00 |
+------------+---------------+-------------+
2 rows in set (0.01 sec)
```

```
mysql> SELECT Department,
    ->        SUM(Amount) AS TotalAmount
    -> FROM Sales
    -> GROUP BY Department
    -> HAVING SUM(Amount) > 50000;
+------------+-------------+
| Department | TotalAmount |
+------------+-------------+
| Electronics |    92300.00 |
+------------+-------------+
1 row in set (0.01 sec)
```

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

Sol:

```
mysql> CREATE TABLE Employee (
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(50) NOT NULL,
    ->     Salary DECIMAL(10,2),
    ->     DeptID INT
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Employee (EmpID, EmpName, Salary, DeptID)
    -> VALUES
    -> (101,'John',50000,1),
    -> (102,'Alice',65000,2),
    -> (103,'David',45000,1),
    -> (104,'Emma',70000,3),
    -> (105,'Sophia',60000,2),
    -> (106,'Michael',40000,3);
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

Corelated Subquery:

```
mysql> SELECT EmpID, EmpName, Salary, DeptID
    -> FROM Employee e
    -> WHERE Salary >
    -> (
    ->     SELECT AVG(Salary)
    ->     FROM Employee
    ->     WHERE DeptID = e.DeptID
    -> );
+-------+---------+----------+--------+
| EmpID | EmpName | Salary   | DeptID |
+-------+---------+----------+--------+
|   101 | John    | 50000.00 |      1 |
|   102 | Alice   | 65000.00 |      2 |
|   104 | Emma    | 70000.00 |      3 |
+-------+---------+----------+--------+
3 rows in set (0.01 sec)
```

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

Sol: